Assignment 3

Huy Cao

2026-05-07

```r
library(DBI)
```

```
## Warning: package 'DBI' was built under R version 4.4.3
```

```r
library(RSQLite)
```

```
## Warning: package 'RSQLite' was built under R version 4.4.3
```

```r
db_path <- "Stats.StackExchange.db"
con <- dbConnect(SQLite(), dbname = db_path)
```

```
SELECT name FROM sqlite_master WHERE type='table';
```

Table 1: Displaying records 1 - 10

name
Badges
Comments
PostHistory
PostLinks
Posts
Users
Votes
TagPosts
PostTypeIdMap
BadgeClassMap

```
PRAGMA table_info(PostTypeIdMap);
```

Table 2: 2 records

cid	name	type	notnull	dflt_value	pk
0	id	INTEGER	0	NA	0
1	value	TEXT	0	NA	0

Question 1: What is the date range for the questions and answers in this database?

Interpretation: "Questions and answers" means rows in the Posts table. Stack Exchange uses PostTypeId to tell post types apart, where 1 is a question and 2 is an answer. Other values like wiki and tag wiki exist in this database (Q5 shows that), but they're not what's being asked about, so I leave them out. For "date" I use CreationDate, which is when the post was first submitted. LastActivityDate and LastEditDate would tell you when a post was last touched, which is a different question. "Date range" just means the earliest and latest CreationDate. I report questions and answers separately as well as combined, partly as a sanity check, since the first answer shouldn't come before the first question.

Reasoning: Filter Posts to PostTypeId IN (1, 2), then take MIN and MAX of CreationDate grouped by post type. A UNION ALL row adds the combined range. SQLite stores dates as text strings in the format "2009-02-02T14:21:12.103". Because the year, month, day, and time appear in that order, sorting these strings as plain text gives the same result as sorting them as real dates, so MIN and MAX work correctly without converting them to a date type first.

```sql
SELECT
  CASE PostTypeId WHEN 1 THEN 'Question' WHEN 2 THEN 'Answer' END AS PostType,
  MIN(CreationDate) AS earliest,
  MAX(CreationDate) AS latest,
  COUNT(*) AS n
FROM Posts
WHERE PostTypeId IN (1, 2)
GROUP BY PostTypeId

UNION ALL

SELECT 'Both', MIN(CreationDate), MAX(CreationDate), COUNT(*)
FROM Posts
WHERE PostTypeId IN (1, 2);
```

```sql
q1
```

```
##   PostType                earliest                  latest      n
## 1 Question 2009-02-02T14:21:12.103 2023-03-05T05:10:18.393 204370
## 2   Answer 2009-02-02T14:24:31.740 2023-03-05T04:48:34.853 197928
## 3     Both 2009-02-02T14:21:12.103 2023-03-05T05:10:18.393 402298
```

Results: The database covers a span from February 2, 2009 to March 5, 2023, which is roughly 14 years of activity. There are 204,370 questions and 197,928 answers, for a combined total of 402,298 posts of these two types. The first question and first answer were posted on the same day, within minutes of each other. This is because Cross Validated was seeded with content from its Area 51 proposal phase when it launched, so the earliest questions and answers were imported together rather than written from scratch on the site.

The latest dates for both types fall on March 5, 2023, which is the cutoff of this particular database dump rather than the end of activity on the site. There are slightly more questions than answers in the database, which lines up with what Q6 shows: about a third of questions never get an answer. The gap is small in relative terms though, only about 3 percent, which suggests that when questions do get answered they usually get only one answer. Q8 confirms this with a median of 1 answer per question.

Question 2: How many users are there? How many users joined since 2020? How many users joined each year? Describe this with a plot.

Interpretatio: All three sub-questions are about the Users table. A "user" is one row in Users, identified by Id. "Joined" maps to CreationDate, the timestamp when the account was created. For "since 2020" I use CreationDate >= '2020-01-01', which includes all of 2020 onward. "Each year" means grouping by the calendar year pulled out of CreationDate.

Reasoning: Three separate queries: a total count, a filtered count for users who joined in 2020 or later, and a grouped count by year. To extract the year from the CreationDate text, I use SQLite's strftime('%Y', ...) function and cast it to an integer so the plot's x-axis treats it as a number rather than a category. A bar chart shows the per-year counts.

```
SELECT COUNT(*) AS n_users FROM Users;
```

```
SELECT COUNT(*) AS n_users_since_2020
FROM Users
WHERE CreationDate >= '2020-01-01';
```

```
SELECT
  CAST(strftime('%Y', CreationDate) AS INTEGER) AS year,
  COUNT(*) AS n_users
FROM Users
WHERE CreationDate IS NOT NULL
GROUP BY year
ORDER BY year;
```

```
q2_total
```

```
##   n_users
## 1  321677
```

```
q2_since2020
```

```
##   n_users_since_2020
## 1             100796
```

```
q2_by_year
```

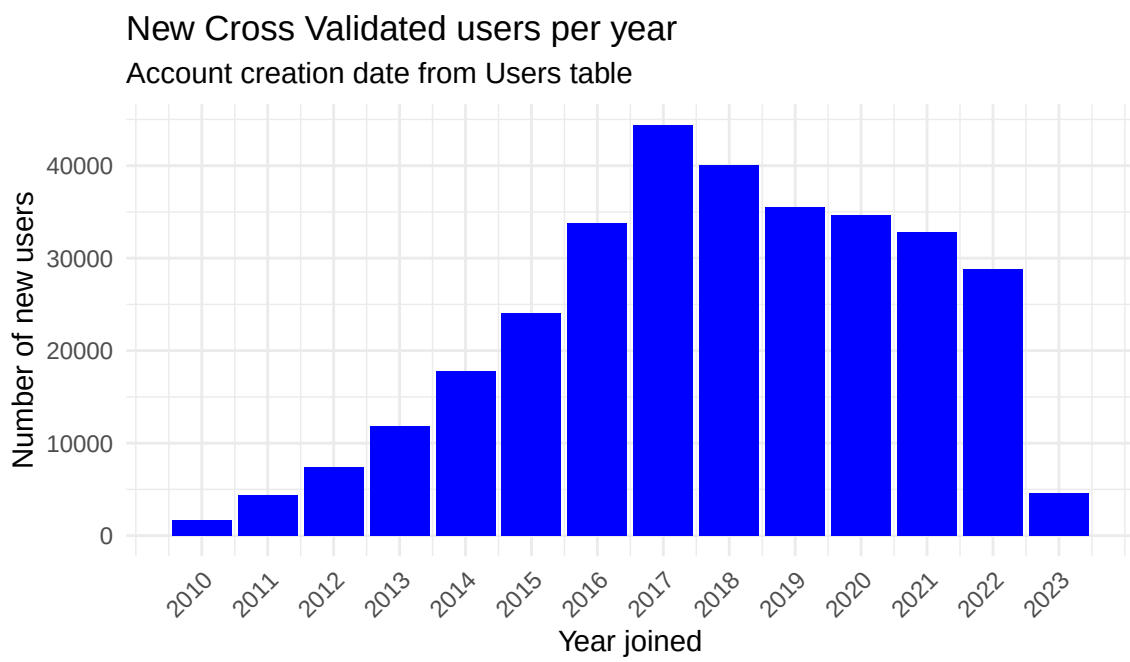
```
##    year n_users
## 1  2010    1668
## 2  2011    4396
## 3  2012    7450
## 4  2013   11846
## 5  2014   17809
## 6  2015   24012
## 7  2016   33753
## 8  2017   44416
## 9  2018   40040
```

```
## 10 2019   35491
## 11 2020   34617
## 12 2021   32765
## 13 2022   28801
## 14 2023    4613
```

```r
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```r
ggplot(q2_by_year, aes(x = year, y = n_users)) +
  geom_col(fill = "blue") +
  scale_x_continuous(breaks = q2_by_year$year) +
  labs(
    title = "New Cross Validated users per year",
    subtitle = "Account creation date from Users table",
    x = "Year joined",
    y = "Number of new users"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Results: The database contains 321,677 user accounts in total. Of these, 100,796 joined in 2020 or later, which is about 31 percent of all users in a span of just over three years out of the site's 14-year history. So while the site has been around since 2010, roughly a third of its current user base signed up in the most recent third of its lifetime.

The per-year breakdown shows steady growth from 2010 through 2017, when new signups peaked at 44,416. After that the count drops off year over year, falling to 28,801 in 2022. This pattern is common for mature Q&A sites: early years bring in users at an increasing rate as the site builds momentum and search traffic, but eventually it slows down.

The 2023 bar (4,613 users) looks artificially small because the database cuts off in early March of that year, so it represents only about two months rather than a full year.

Question 3: What user answered the most questions?

Interpretation: An "answer" is a row in the Posts table with PostTypeId = 2. The "user who answered" is identified by the OwnerUserId column on that answer row, which is the account credited as the author. "Answered the most" means the user with the highest count of answer rows. I return the top user along with nine runners-up, partly for context and partly to show the shape of the distribution at the top. I join to the Users table to recover the display name and reputation.

A couple of edge cases worth noting. Some answers have OwnerUserId set to NULL, which happens when the author's account has been deleted. Those rows can't be tied to a current user, so I leave them out. The Community user with Id = -1 is also excluded since it's an automated account that takes ownership of posts whose original author was removed, not a real person.

Reasoning: Filter Posts to PostTypeId = 2, group by OwnerUserId, count rows, sort descending, and join to Users to pull in the display name and reputation. LIMIT 10 gives the top user (the direct answer to the question) plus nine more for comparison.

```sql
SELECT
  u.Id AS user_id,
  u.DisplayName AS display_name,
  u.Reputation AS reputation,
  COUNT(*) AS n_answers
FROM Posts p
JOIN Users u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 2
  AND p.OwnerUserId IS NOT NULL
  AND p.OwnerUserId != -1
GROUP BY u.Id
ORDER BY n_answers DESC
LIMIT 10;
```

```
q3
```

```
##    user_id       display_name reputation n_answers
## 1      805             Glen_b     268966      4733
## 2      919             whuber     304878      3051
## 3      686         Peter Flom      97091      2875
## 4    28500                EdM      76431      2703
## 5    35989                Tim     126510      2454
## 6    11887 kjetil b halvorsen      70519      2371
## 7   173082                Ben     108435      2240
## 8    85665            BruceET      53074      2189
## 9     1352    Stephan Kolassa     107905      2124
## 10  204068              gunes      54443      1873
```

Results: The user who answered the most questions is Glen_b with 4,733 answers. That is a substantial lead over the rest of the top 10: the second-place answerer, whuber, has 3,051 answers, which is roughly 65 percent of Glen_b's total. The drop continues steeply, with the tenth place user (gunes) at 1,873 answers, less than 40 percent of Glen_b's count.

The reputations of the top answerers are also extremely high, all above 50,000 and topping out at whuber with 304,878. Since reputation on Stack Exchange is earned primarily through upvotes on answers.

Question 4: How many users have asked at least one question? How many have only answered? Only commented? Not posted anything?

Interpretation: Four user categories defined by posting behavior. The first ("asked at least one question") is not mutually exclusive with the others, since a user who has asked a question can also have answered or commented. The next three are exclusive: "only answered" means a user has at least one answer but zero questions and zero comments, and so on. "Not posted anything" means zero questions, zero answers, and zero comments.

A user is identified as the author of a post through OwnerUserId in the Posts table and UserId in the Comments table. I exclude NULL and -1 (the Community placeholder) on the post side when matching authorship, since those don't correspond to real user accounts. Every row in the Users table is still counted in the total though, since the question is about users.

Reasoning: The cleanest way to handle this is to build per-user flags for whether they have ever asked a question, ever answered, and ever commented. Then a single aggregation counts users in each combination. I use three CTEs (WITH clauses) to first build per-user counts of each activity, then a fourth to attach 0 or 1 flags to every user in the Users table, then a final SELECT that sums up the categories. The LEFT JOIN from Users to those counts is what makes the "no posts" group countable: users who never appear in any of the activity tables still have rows in the flags table, with all three flags set to 0.

```sql
WITH
q AS (
  SELECT OwnerUserId AS uid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 1 AND OwnerUserId IS NOT NULL AND OwnerUserId != -1
  GROUP BY OwnerUserId
),
a AS (
  SELECT OwnerUserId AS uid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2 AND OwnerUserId IS NOT NULL AND OwnerUserId != -1
  GROUP BY OwnerUserId
),
c AS (
  SELECT UserId AS uid, COUNT(*) AS n
  FROM Comments
  WHERE UserId IS NOT NULL AND UserId != -1
  GROUP BY UserId
),
flags AS (
  SELECT
    u.Id AS uid,
    CASE WHEN q.n > 0 THEN 1 ELSE 0 END AS asked,
    CASE WHEN a.n > 0 THEN 1 ELSE 0 END AS answered,
    CASE WHEN c.n > 0 THEN 1 ELSE 0 END AS commented
  FROM Users u
  LEFT JOIN q ON q.uid = u.Id
  LEFT JOIN a ON a.uid = u.Id
  LEFT JOIN c ON c.uid = u.Id
)
SELECT
  SUM(asked) AS asked_at_least_one,
```

```sql
  SUM(CASE WHEN answered = 1 AND asked = 0 AND commented = 0 THEN 1 ELSE 0 END) AS only_answered,
  SUM(CASE WHEN commented = 1 AND asked = 0 AND answered = 0 THEN 1 ELSE 0 END) AS only_commented,
  SUM(CASE WHEN asked = 0 AND answered = 0 AND commented = 0 THEN 1 ELSE 0 END) AS no_posts,
  COUNT(*) AS total_users
FROM flags;
```

```
q4
```

```
##   asked_at_least_one only_answered only_commented no_posts total_users
## 1              88923         10263           5009   213257      321677
```

Results. Out of 321,677 total user accounts, 88,923 (about 28 percent) have asked at least one question. Only 10,263 users have answered without ever asking or commenting, and only 5,009 have commented without ever asking or answering. By far the largest group is users who have never posted anything at all: 213,257 accounts, which is about 66 percent of all registered users.

Question 5: How many different types of posts are there in the Posts table? Get the description of the types from the PostTypes table.

Interpretation: The Posts table has a PostTypeId column whose values are codes that map to descriptive labels in a separate lookup table. The assignment refers to this lookup as "PostTypes," but in this database it is named PostTypeIdMap, with columns id (the integer code) and value (the text description). "How many different types" means counting distinct PostTypeId values that actually appear in Posts, not all possible types defined in the lookup. I also include the count of posts for each type, since it gives useful context for what the post types actually represent in practice.

Reasoning: Group Posts by PostTypeId and count rows, then LEFT JOIN to PostTypeIdMap to attach the descriptive label. The number of rows the query returns is the direct answer to "how many different types." I use LEFT JOIN rather than INNER JOIN as a defensive choice: if a PostTypeId in Posts didn't have a matching row in the lookup, the LEFT JOIN would still show that row with a NULL label rather than silently dropping it.

```sql
SELECT
  p.PostTypeId,
  pt.value AS post_type,
  COUNT(*) AS n_posts
FROM Posts p
LEFT JOIN PostTypeIdMap pt ON pt.id = p.PostTypeId
GROUP BY p.PostTypeId
ORDER BY n_posts DESC;
```

```
q5
```

```
##   PostTypeId                                                 post_type
## 1          1                                                  Question
## 2          2                                                    Answer
## 3          5                                                  Tag wiki
## 4          4                                         Tag wiki excerpt
## 5          6                                       Moderator nomination
```

```
## 6          3                                          Orphaned tag wiki
## 7          7 Wiki placeholder (seems to only be the election description)
##   n_posts
## 1  204370
## 2  197928
## 3    1444
## 4    1444
## 5      23
## 6       6
## 7       5
```

```r
nrow(q5)
```

```
## [1] 7
```

Results: There are 7 different post types present in the database. Questions (204,370) and answers (197,928) account for nearly all of them, more than 99 percent of all posts. he remaining five types are much rarer and are mostly behind-the-scenes content rather than the user-facing Q&A flow: *Tag wikis and tag wiki excerpts (1,444 each) are the descriptive blurbs that show up next to tags. The matching counts make sense because they appear in pairs (one wiki and one excerpt per tag).* Moderator nominations (23) come from the elections that pick new moderators for the site. The count is small because elections are infrequent. *Orphaned tag wikis (6) are tag wikis whose tag was renamed or removed.* Wiki placeholders (5) are described in the lookup table as election descriptions, also tied to moderator elections.

Question 6: How long does it take to get the first response to a question? The accepted answer? The last response? How many questions received no response, and what percentage is this of all posts?

Interpretation: A "response" I interpret as an answer (a row in Posts with PostTypeId = 2), not a comment. Comments are often clarifying questions back to the asker rather than actual responses, and the question groups "response" together with "accepted answer," which is unambiguously an answer. The "accepted answer" is the answer whose Id matches the question's AcceptedAnswerId field. Not every question has one. The "last response" is the most recent answer to a question, by CreationDate. "No response" means a question that has zero answers. To capture these I use a LEFT JOIN from questions to answers, which keeps unanswered questions in the result rather than dropping them. For "time to" I compute the answer's CreationDate minus the question's CreationDate. SQLite has a function julianday() that converts a date string to a fractional day number, so subtracting two julianday values gives a difference in days. I multiply by 24 to get hours, which is more readable for the typical scale of these times. For "percentage of all posts" I interpret "all posts" as the count of all rows in the Posts table (any type). I also report the percentage of questions, since that's arguably the more natural denominator and gives a different perspective.

Reasoning: I build a per-question table where each row has the time differences to the first answer, accepted answer, and last answer. The query uses three CTEs: one for the question metadata, one that aggregates min and max answer dates per question, and one that pulls the accepted answer's date through the AcceptedAnswerId link. LEFT JOINs preserve questions without answers (those rows just get NA for the time columns). A separate query handles the no-response count: I use a LEFT JOIN to a list of questions that have answers, then count questions where the join didn't match.

```sql
WITH questions AS (
  SELECT Id, CreationDate, AcceptedAnswerId
  FROM Posts
  WHERE PostTypeId = 1
),
answer_stats AS (
  SELECT
    ParentId AS qid,
    MIN(CreationDate) AS first_answer_date,
    MAX(CreationDate) AS last_answer_date
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
accepted AS (
  SELECT Id AS aid, CreationDate AS accepted_date
  FROM Posts
  WHERE PostTypeId = 2
)
SELECT
  (julianday(a.first_answer_date) - julianday(q.CreationDate)) * 24 AS hours_to_first,
  (julianday(acc.accepted_date)   - julianday(q.CreationDate)) * 24 AS hours_to_accepted,
  (julianday(a.last_answer_date)  - julianday(q.CreationDate)) * 24 AS hours_to_last
FROM questions q
LEFT JOIN answer_stats a   ON a.qid = q.Id
LEFT JOIN accepted    acc  ON acc.aid = q.AcceptedAnswerId;
```

```r
summary(q6_times)
```

```
##  hours_to_first      hours_to_accepted  hours_to_last
##  Min.   :-14964.90   Min.   :   -1.76   Min.   :    -7.09
##  1st Qu.:     0.77   1st Qu.:    0.86   1st Qu.:     1.32
##  Median :     3.11   Median :    3.39   Median :     7.37
##  Mean   :  1306.30   Mean   :  607.35   Mean   :  3767.99
##  3rd Qu.:    25.36   3rd Qu.:   22.06   3rd Qu.:   118.08
##  Max.   :100732.06   Max.   :93053.85   Max.   :109884.72
##  NA's   :66970       NA's   :136366     NA's   :66970
```

```r
sapply(q6_times, median, na.rm = TRUE)
```

```
##    hours_to_first hours_to_accepted     hours_to_last
##          3.109062          3.385504          7.368921
```

```r
sapply(q6_times, mean,   na.rm = TRUE)
```

```
##    hours_to_first hours_to_accepted     hours_to_last
##         1306.3042          607.3489         3767.9928
```

```sql
WITH questions AS (
  SELECT Id FROM Posts WHERE PostTypeId = 1
),
```

```sql
answered AS (
  SELECT DISTINCT ParentId AS qid FROM Posts WHERE PostTypeId = 2
)
SELECT
  (SELECT COUNT(*) FROM questions) AS total_questions,
  (SELECT COUNT(*) FROM questions q LEFT JOIN answered a ON a.qid = q.Id
     WHERE a.qid IS NULL) AS questions_no_response,
  (SELECT COUNT(*) FROM Posts) AS total_posts;
```

```r
q6_noresponse
```

```
##   total_questions questions_no_response total_posts
## 1          204370                 66970      405220
```

```r
q6_noresponse$pct_of_all_posts <- 100 * q6_noresponse$questions_no_response / q6_noresponse$total_posts
q6_noresponse$pct_of_questions <- 100 * q6_noresponse$questions_no_response / q6_noresponse$total_quest
q6_noresponse
```

```
##   total_questions questions_no_response total_posts pct_of_all_posts
## 1          204370                 66970      405220         16.52682
##   pct_of_questions
## 1           32.769
```

Results: The median time to the first answer is 3.1 hours. The median time to the accepted answer is 3.4 hours. The median time to the last answer is 7.4 hours.

The mean values are much larger: 1,306 hours (about 54 days) to first answer, 607 hours (about 25 days) to accepted answer, and 3,768 hours (about 157 days) to last answer. The gap between medians and means reflects extreme right-skew in the distributions, with max values exceeding 100,000 hours.

There are a few negative values in the time columns (minimum of -14,964 hours for hours_to_first). These come from posts imported from the site's Area 51 phase with clock or migration artifacts, and are too few to affect the medians.

66,970 questions out of 204,370 received no answer at all, which is 32.8 percent of all questions. As a percentage of all 405,220 posts in the database, unanswered questions make up 16.5 percent.

Question 7: What are the top 50 most common tags on questions? For each of the top 50 tags on questions, how many questions are there for each tag?

Interpretation: The assignment notes that the TagPosts table contains one row per tag per question, with Id holding the question id and Tag holding the tag name (without the angle brackets that appear in the Tags field on Posts). So the count of a tag is the number of rows in TagPosts with that tag, which equals the number of questions carrying it.

I also join to Posts and filter on PostTypeId = 1 as a defensive check, to confirm the linked posts are actually questions. Tags on Stack Exchange only apply to questions, so every row in TagPosts should already point to a question, but the filter guards against any quirks in the data dump.

Reasoning: Join TagPosts to Posts on the question id, filter to questions, group by tag, count, sort descending, and take the top 50

```sql
SELECT
  tp.Tag AS tag,
  COUNT(*) AS n_questions
FROM TagPosts tp
JOIN Posts p ON p.Id = tp.Id
WHERE p.PostTypeId = 1
GROUP BY tp.Tag
ORDER BY n_questions DESC
LIMIT 50;
```

```
q7
```

```
##                          tag n_questions
## 1                          r       28495
## 2                 regression       28146
## 3           machine-learning       19355
## 4                time-series       13745
## 5                probability       11894
## 6         hypothesis-testing       10091
## 7              distributions        9147
## 8                 self-study        7985
## 9            neural-networks        7793
## 10                  bayesian        7628
## 11                  logistic        7507
## 12  mathematical-statistics        7455
## 13            classification        6654
## 14               correlation        6074
## 15 statistical-significance        6038
## 16      normal-distribution        5877
## 17               mixed-model        5837
## 18      multiple-regression        5265
## 19                     anova        5100
## 20                    python        4605
## 21      confidence-interval        4367
## 22 generalized-linear-model        4276
## 23                  variance        4042
## 24                clustering        3932
## 25               forecasting        3726
## 26                    t-test        3486
## 27          categorical-data        3468
## 28          cross-validation        3385
## 29                       pca        3333
## 30       maximum-likelihood        3209
## 31                estimation        3159
## 32                 lme4-nlme        3156
## 33                  sampling        3104
## 34         predictive-models        2971
## 35                  survival        2960
## 36       data-visualization        2949
## 37                 inference        2899
## 38                     arima        2818
## 39                   p-value        2709
## 40                      mean        2705
```

```
## 41             optimization        2688
## 42            least-squares        2682
## 43        repeated-measures        2612
## 44         chi-squared-test        2559
## 45                 modeling        2501
## 46               references        2451
## 47    multivariate-analysis        2430
## 48             econometrics        2422
## 49              interaction        2421
## 50             linear-model        2412
```

Results: The most common tag by a slim margin is "r" with 28,495 questions, followed closely by "regression" with 28,146. After that the counts drop more substantially, with "machine-learning" at 19,355 in third place and "time-series" at 13,745 in fourth.

The top 10 tags cover the core topics of the site: r, regression, machine-learning, time-series, probability, hypothesis-testing, distributions, self-study, neural-networks, and bayesian, all with 7,000 or more questions each. The 50th most common tag, "linear-model," still has 2,412 questions, showing that even outside the top tier there is a substantial volume of tagged content.

Two of the top 50 tags are programming languages, "r" (rank 1) and "python" (rank 20), reflecting that a large share of statistical questions come with implementation context. The other 48 are statistical concepts, methods, or model classes.

#Question 8: What is the distribution of the number of responses to questions? And the number of comments for each question?

Interpretation: "Number of responses" is the number of answers (rows in Posts with PostTypeId = 2 linked to the question via ParentId). As in Q6, I treat "response" as answer, not comment. "Number of comments" is the number of rows in Comments with PostId equal to the question's Id. Comments can exist on both questions and answers; here I only count comments on the question itself, since the question asks about comments "for each question." Questions with zero answers (or zero comments) are part of the distribution and need to be preserved. LEFT JOIN handles this so they show up with count zero rather than being dropped. "Distribution" I interpret as the per-question counts themselves (one row per question), plus summary statistics. The raw counts are what let me compute quantiles, ranges, and frequency tables in R.

Reasoning: I build per-question counts using LEFT JOINs to answer and comment aggregates. COALESCE turns NULL counts from missing joins into 0. Then I summarize with summary(), frequency tables for the low end where most questions fall, and quantiles for the long tail.

```sql
WITH questions AS (
  SELECT Id FROM Posts WHERE PostTypeId = 1
),
answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
comment_counts AS (
  SELECT PostId AS qid, COUNT(*) AS n
  FROM Comments
  GROUP BY PostId
)
SELECT
  q.Id AS question_id,
```

```sql
  COALESCE(a.n, 0) AS n_answers,
  COALESCE(c.n, 0) AS n_comments
FROM questions q
LEFT JOIN answer_counts  a ON a.qid = q.Id
LEFT JOIN comment_counts c ON c.qid = q.Id;
```

```r
summary(q8[, c("n_answers", "n_comments")])
```

```
##    n_answers          n_comments
##  Min.   :  0.0000   Min.   : 0.000
##  1st Qu.:  0.0000   1st Qu.: 0.000
##  Median :  1.0000   Median : 1.000
##  Mean   :  0.9685   Mean   : 2.171
##  3rd Qu.:  1.0000   3rd Qu.: 3.000
##  Max.   :153.0000   Max.   :54.000
```

```r
table(pmin(q8$n_answers, 10))
```

```
##
##     0     1     2     3     4     5     6     7     8     9    10
## 66970 98602 27191  7246  2408   905   401   210   136    82   219
```

```r
table(pmin(q8$n_comments, 10))
```

```
##
##     0     1     2     3     4     5     6     7     8     9    10
## 76090 30707 30417 20645 14897  9832  6806  4680  3236  2158  4902
```

```r
quantile(q8$n_answers,  probs = c(0.5, 0.75, 0.9, 0.95, 0.99, 1))
```

```
##  50%  75%  90%  95%  99% 100%
##    1    1    2    3    4  153
```

```r
quantile(q8$n_comments, probs = c(0.5, 0.75, 0.9, 0.95, 0.99, 1))
```

```
##  50%  75%  90%  95%  99% 100%
##    1    3    6    8   12   54
```

Results: For answers per question, the median is 1, the mean is 0.97, and the max is 153. The frequency table shows that 66,970 questions have 0 answers, 98,602 have exactly 1, and 27,191 have 2. After that the counts drop off quickly: only 219 questions have 10 or more answers. The 95th percentile is 3 answers and the 99th percentile is 4.

For comments per question, the median is 1, the mean is 2.17, and the max is 54. The distribution is more spread out than answers: 76,090 questions have 0 comments, 30,707 have 1, 30,417 have 2, and the count stays above 4,000 even at 7 comments. The 95th percentile is 8 comments and the 99th percentile is 12.

Both distributions are heavily right-skewed, but comments are more abundant and spread further into the tail than answers. The mean comments per question (2.17) is more than double the mean answers per question (0.97), and a substantial fraction of questions accumulate many comments while never getting more than one or two formal answers.

Question 9: How many questions are there each year?

Interpretation: A question is a row in Posts with PostTypeId = 1. "Each year" means the calendar year pulled out of CreationDate. The result is one row per year with the count of questions created that year.

Reasoning: Filter Posts to questions, extract the year from CreationDate using SQLite's strftime('%Y', ...) function, cast to integer so it sorts and prints as a number, group by year, count, and sort.

```
SELECT
  CAST(strftime('%Y', CreationDate) AS INTEGER) AS year,
  COUNT(*) AS n_questions
FROM Posts
WHERE PostTypeId = 1
  AND CreationDate IS NOT NULL
GROUP BY year
ORDER BY year;
```

```
q9
```

```
##    year n_questions
## 1  2009           3
## 2  2010        1477
## 3  2011        4935
## 4  2012        8259
## 5  2013       11620
## 6  2014       15888
## 7  2015       18665
## 8  2016       20914
## 9  2017       21437
## 10 2018       19920
## 11 2019       19744
## 12 2020       21108
## 13 2021       17271
## 14 2022       19651
## 15 2023        3478
```

Results: The earliest year, 2009, has only 3 questions, which corresponds to the small set of seed posts from the Area 51 phase. From 2010 onward the site shows real activity, starting at 1,477 questions and growing each year through 2017, which is the peak at 21,437 questions.

After 2017 the question volume levels off and then slowly declines, with 19,920 in 2018, 19,744 in 2019, a small bump to 21,108 in 2020, and then 17,271 in 2021. 2022 comes back up to 19,651. The 2023 count of 3,478 only reflects the first two months of the year, so it isn't directly comparable to the full-year totals.

The overall pattern matches the user signup trajectory from Q2: rapid growth through the mid-2010s, a peak around 2017, and then a plateau with mild year-to-year fluctuation.

Question 10: How many posts are there per question per year? Draw the distribution of the number of posts per question for each year (e.g., density plots, boxplots, violin plots). Has the distribution changed over time?

Interpretation: "Posts per question" I interpret as the total number of Posts table rows tied to a question: the question itself plus its answers (PostTypeId = 2, linked via ParentId). So for each question, the count is 1 + number_of_answers. Comments are not in the Posts table (they're in a separate Comments table), so they're not "posts" in the sense the table name implies.

"Per year" groups questions by the year of the question's own CreationDate, not the year of the answers. So a 2015 question that got an answer added in 2020 still counts under 2015. This is the natural grouping for describing the distribution of activity attached to each question.

Reasoning: Build a per-question table with the question's year and the count 1 + n_answers. LEFT JOIN to an answer-count CTE so questions with zero answers get posts_per_question = 1. Then summarize medians, quartiles, max, and means by year, and plot the per-year distributions with a violin + boxplot combination on a log y-axis to handle the right-skew.

```sql
WITH questions AS (
  SELECT
    Id,
    CAST(strftime('%Y', CreationDate) AS INTEGER) AS year
  FROM Posts
  WHERE PostTypeId = 1
    AND CreationDate IS NOT NULL
),
answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
)
SELECT
  q.Id AS question_id,
  q.year,
  1 + COALESCE(a.n, 0) AS posts_per_question
FROM questions q
LEFT JOIN answer_counts a ON a.qid = q.Id;
```

```r
# Summary by year: medians and quartiles
aggregate(posts_per_question ~ year, data = q10,
          FUN = function(x) c(
            median = median(x),
            q75    = quantile(x, 0.75, names = FALSE),
            q95    = quantile(x, 0.95, names = FALSE),
            max    = max(x)
          ))
```

```
##    year posts_per_question.median posts_per_question.q75 posts_per_question.q95
## 1  2009                      8.0                   12.0                   15.2
## 2  2010                      3.0                    5.0                   11.0
## 3  2011                      3.0                    3.0                    6.0
```

```
## 4  2012                      2.0                     3.0                     5.0
## 5  2013                      2.0                     3.0                     4.0
## 6  2014                      2.0                     3.0                     4.0
## 7  2015                      2.0                     2.0                     4.0
## 8  2016                      2.0                     2.0                     4.0
## 9  2017                      2.0                     2.0                     3.0
## 10 2018                      2.0                     2.0                     3.0
## 11 2019                      2.0                     2.0                     3.0
## 12 2020                      2.0                     2.0                     3.0
## 13 2021                      2.0                     2.0                     3.0
## 14 2022                      1.0                     2.0                     3.0
## 15 2023                      1.0                     2.0                     3.0
##    posts_per_question.max
## 1                    16.0
## 2                   154.0
## 3                    23.0
## 4                    35.0
## 5                    16.0
## 6                    22.0
## 7                    20.0
## 8                    17.0
## 9                    22.0
## 10                   17.0
## 11                   14.0
## 12                   33.0
## 13                   14.0
## 14                   12.0
## 15                    8.0
```

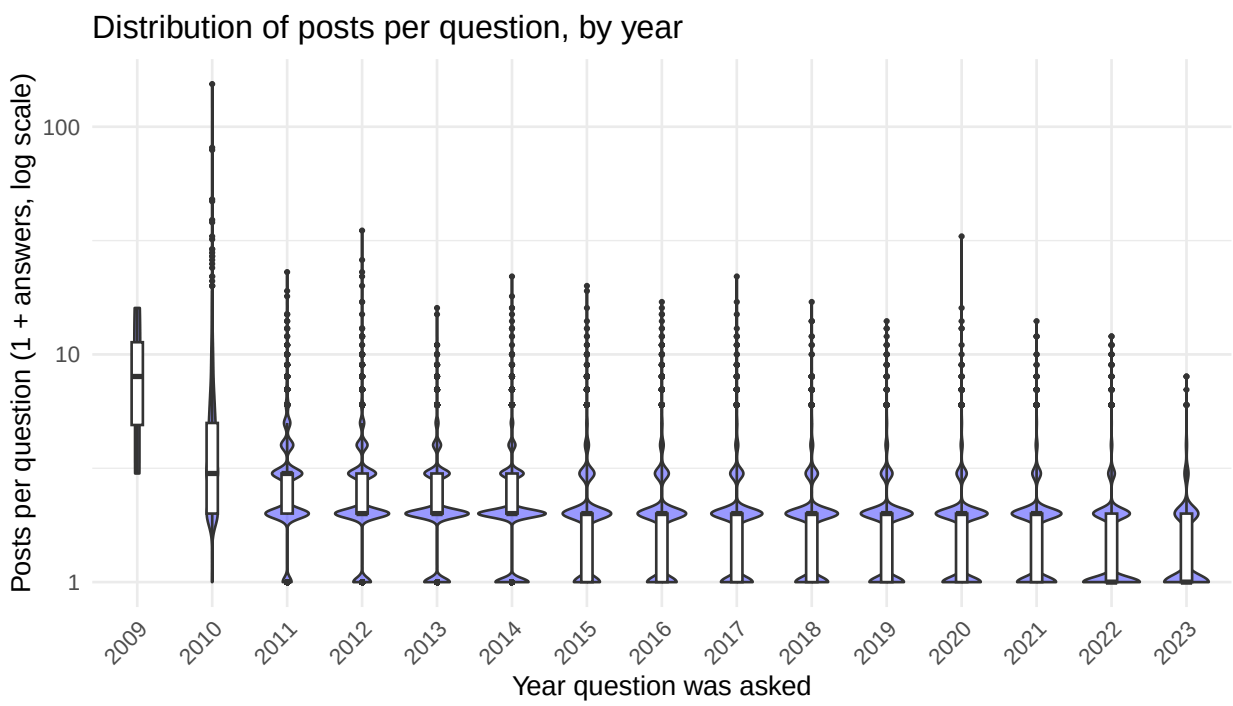
```r
# Mean alongside the median given the skew
aggregate(posts_per_question ~ year, data = q10, FUN = mean)
```

```
##    year posts_per_question
## 1  2009           9.000000
## 2  2010           4.712255
## 3  2011           2.906991
## 4  2012           2.553094
## 5  2013           2.262134
## 6  2014           2.140609
## 7  2015           2.048486
## 8  2016           1.983886
## 9  2017           1.926529
## 10 2018           1.889809
## 11 2019           1.855602
## 12 2020           1.798560
## 13 2021           1.760292
## 14 2022           1.561905
## 15 2023           1.500000
```

```r
library(ggplot2)

ggplot(q10, aes(x = factor(year), y = posts_per_question)) +
  geom_violin(fill = "blue", alpha = 0.4) +
```

```r
  geom_boxplot(width = 0.15, outlier.size = 0.5) +
  scale_y_log10() +
  labs(
    title = "Distribution of posts per question, by year",
    x = "Year question was asked",
    y = "Posts per question (1 + answers, log scale)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Results: Yes, the distribution has changed over time, and the direction is clear: posts per question has been declining steadily since the early years of the site.

The 2009 seed posts have a median of 8 posts per question, but that's a tiny sample (3 questions) and doesn't represent normal site behavior. 2010 starts at a median of 3 with a 75th percentile of 5 and a 95th percentile of 11. By 2012 the median drops to 2 with a 75th percentile of 3. From 2013 onward the median holds at 2 through 2021, and the 95th percentile drops from 4 to 3 along the way. By 2022 the median falls to 1.

The mean tells the same story more smoothly: 4.71 in 2010, 2.91 in 2011, 2.55 in 2012, then a gradual decline through every year, reaching 1.56 in 2022 and 1.50 in 2023. So an average question in 2010 had nearly 5 posts attached to it (one question and almost 4 answers), while an average question in 2022 has barely more than 1 post total (the question itself and a less-than-even chance of any answer).

The max column shows that extreme cases still exist in every year, with at least one question receiving 12 or more posts even in recent years, but they are rare outliers against a backdrop of mostly unanswered or once-answered questions.

Question 11: How many questions have each of the following terms: Neural network, classification tree, ANOVA, boosting, bagging, clustering, multiple testing?

Interpretation: "Have the term" I interpret as the term appearing in the question's text, meaning either the Title or the Body of the question. Searching only the title would undercount because titles are short and don't always name every concept the body discusses. Searching only Tags would miss questions that use the term in prose but were tagged differently or not at all. Title OR Body is the broadest natural reading.

Matching is case-insensitive and uses substring matching with LIKE '%term%'. Case-insensitivity is essential since "ANOVA," "anova," and "Anova" all refer to the same thing. SQLite's LIKE is case-insensitive for ASCII characters by default, which covers all the terms here. Substring matching means "boosting" will also match "gradient boosting," and "neural network" will catch "neural networks" via the trailing wildcard. This is intended behavior for "questions about boosting" but means the counts include closely related variants.

Each term is counted independently, so a question containing both "ANOVA" and "clustering" contributes to both counts. The totals therefore won't sum to a unique question count.

Reasoning: One query that scans Posts filtered to questions and uses a SUM(CASE WHEN ... LIKE ... THEN 1 ELSE 0 END) per term. Doing all terms in a single query is faster than seven separate queries since the Body column (which contains the full HTML of the question) only needs to be scanned once.

```
SELECT
  SUM(CASE WHEN Title LIKE '%neural network%' OR Body LIKE '%neural network%' THEN 1 ELSE 0 END) AS neu
  SUM(CASE WHEN Title LIKE '%classification tree%' OR Body LIKE '%classification tree%' THEN 1 ELSE 0 E
  SUM(CASE WHEN Title LIKE '%anova%' OR Body LIKE '%anova%' THEN 1 ELSE 0 END) AS anova,
  SUM(CASE WHEN Title LIKE '%boosting%' OR Body LIKE '%boosting%' THEN 1 ELSE 0 END) AS boosting,
  SUM(CASE WHEN Title LIKE '%bagging%' OR Body LIKE '%bagging%' THEN 1 ELSE 0 END) AS bagging,
  SUM(CASE WHEN Title LIKE '%clustering%' OR Body LIKE '%clustering%' THEN 1 ELSE 0 END) AS clustering,
  SUM(CASE WHEN Title LIKE '%multiple testing%' OR Body LIKE '%multiple testing%' THEN 1 ELSE 0 END) AS
FROM Posts
WHERE PostTypeId = 1;
```

```
q11
```

```
##   neural_network classification_tree anova boosting bagging clustering
## 1           5742                 164  7431      861     297       4197
##   multiple_testing
## 1              393
```

```
# Reshape into a tall table for nicer presentation
data.frame(
  term = names(q11),
  n_questions = as.integer(q11[1, ])
)
```

```
##                  term n_questions
## 1      neural_network        5742
## 2 classification_tree         164
## 3               anova        7431
## 4            boosting         861
## 5             bagging         297
## 6          clustering        4197
## 7    multiple_testing         393
```

Results: ANOVA is the most common term with 7,431 questions, followed by neural network with 5,742 and clustering with 4,197. Boosting (861), multiple testing (393), and bagging (297) are much less common. Classification tree has the lowest count at 164.

The classification tree count is notably small. This is likely because the term competes with related vocabulary that wasn't searched for, like "decision tree," "CART," or just "tree-based method." Users on Cross Validated tend to reach for the more general "decision tree" terminology when writing questions, and the literal phrase "classification tree" is comparatively rare.

The gap between boosting (861) and bagging (297) is also notable. The two are often discussed together as ensemble methods, but boosting (especially gradient boosting and XGBoost) has gotten much more attention in practice over the past decade, and the question counts reflect that imbalance.

Question 12: Using the Posts and PostLinks tables, how many questions gave rise to a "related" or "duplicate" question? And how many responses did these questions get? How experienced were the users posting these questions?

Interpretation: The PostLinks table connects related posts with a LinkTypeId column that maps to the LinkTypeMap lookup. In this database, LinkTypeMap has id = 1 for "Linked (PostId contains a link to RelatedPostId)" and id = 3 for "Duplicate (PostId is a duplicate of RelatedPostId)." Each PostLinks row has a PostId and a RelatedPostId, and the link points from the newer post to the post it duplicates or links back to.

The phrase "gave rise to" matters for the direction. I interpret it as the original question that triggered later duplicates or related links, not the duplicates themselves. So I look for questions that appear as RelatedPostId in a PostLinks row, meaning some later question was marked as related or duplicate to this one. A question that links out to other questions is not what I count here; what I count is questions that have other questions pointing back to them.

"How many responses" is the number of answers, consistent with earlier questions.

"How experienced were the users" I take to mean the asker's Reputation from the Users table, joined via Posts.OwnerUserId. Reputation is cumulative and roughly tracks site tenure plus quality of contributions, so it works as a single number proxy for experience. I report summary statistics rather than a single value, and compare against a baseline of all askers in the database for context.

Reasoning: First, peek at LinkTypeMap to confirm the codes. Then build the set of question ids that appear as RelatedPostId, separated by LinkTypeId, and use DISTINCT to avoid double-counting questions that have many incoming links. Join to Posts and Users to get the asker, and to an answer-count CTE for response counts. A separate baseline query pulls reputation for all askers for comparison.

```sql
SELECT * FROM LinkTypeMap;
```

Table 3: 2 records

id	value
1	Linked (PostId contains a link to RelatedPostId)
3	Duplicate (PostId is a duplicate of RelatedPostId)

```sql
WITH answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n_answers
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
linked_questions AS (
  SELECT DISTINCT
    pl.RelatedPostId AS qid,
    pl.LinkTypeId
  FROM PostLinks pl
  JOIN Posts p ON p.Id = pl.RelatedPostId
  WHERE p.PostTypeId = 1
)
SELECT
  lq.qid AS question_id,
  lq.LinkTypeId,
  p.OwnerUserId,
  u.Reputation AS asker_reputation,
  COALESCE(a.n_answers, 0) AS n_answers
FROM linked_questions lq
JOIN Posts p ON p.Id  = lq.qid
LEFT JOIN Users u ON u.Id  = p.OwnerUserId
LEFT JOIN answer_counts a ON a.qid = lq.qid;
```

```r
q12$link_type <- ifelse(q12$LinkTypeId == 3, "Duplicate", ifelse(q12$LinkTypeId == 1, "Related", as.cha

# A question can appear under both link types, count distinct questions per type and across both.
cat("Distinct questions that gave rise to at least one link, by type:\n")
```

```
## Distinct questions that gave rise to at least one link, by type:
```

```r
aggregate(question_id ~ link_type, data = q12,
          FUN = function(x) length(unique(x)))
```

```
##   link_type question_id
## 1 Duplicate        7037
## 2   Related       29602
```

```r
cat("\nDistinct questions that gave rise to any related OR duplicate link:\n")
```

```
##
## Distinct questions that gave rise to any related OR duplicate link:
```

```r
length(unique(q12$question_id))
```

```
## [1] 30809
```

```r
# Responses received
cat("\nNumber of answers (responses) - summary, by link type:\n")
```

```
##
## Number of answers (responses) - summary, by link type:
```

```r
aggregate(n_answers ~ link_type, data = q12, FUN = summary)
```

```
##   link_type n_answers.Min. n_answers.1st Qu. n_answers.Median n_answers.Mean
## 1 Duplicate       0.000000          1.000000         2.000000       2.326844
## 2   Related       0.000000          1.000000         1.000000       1.799574
##   n_answers.3rd Qu. n_answers.Max.
## 1          3.000000      38.000000
## 2          2.000000     153.000000
```

```r
# Asker experience (reputation), by link type
cat("\nAsker reputation - summary, by link type:\n")
```

```
##
## Asker reputation - summary, by link type:
```

```r
aggregate(asker_reputation ~ link_type, data = q12,
          FUN = function(x) summary(x))
```

```
##   link_type asker_reputation.Min. asker_reputation.1st Qu.
## 1 Duplicate                 1.000                  121.000
## 2   Related                 1.000                  111.000
##   asker_reputation.Median asker_reputation.Mean asker_reputation.3rd Qu.
## 1                 403.000              3195.899                 1392.000
## 2                 343.500              3327.154                 1315.000
##   asker_reputation.Max.
## 1            304878.000
## 2            304878.000
```

```sql
SELECT u.Reputation AS asker_reputation
FROM Posts p
JOIN Users u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1
  AND p.OwnerUserId IS NOT NULL
  AND p.OwnerUserId != -1;
```

```r
cat("Reputation of askers whose questions spawned a link:\n")
```

```
## Reputation of askers whose questions spawned a link:
```

```r
summary(q12$asker_reputation)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##       1     111     355    3302    1336  304878     754
```

```r
cat("\nReputation of all questioners (baseline):\n")
```

```
##
## Reputation of all questioners (baseline):
```

```r
summary(q12_baseline$asker_reputation)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       1      25     131    1299     501  304878
```

Results: 29,602 distinct questions gave rise to a "related" link, and 7,037 gave rise to a "duplicate" link. The two categories overlap (a question can be both linked to and marked a duplicate of by different later questions), and the combined count of distinct questions that gave rise to any related OR duplicate link is 30,809.

Questions that spawned duplicates received slightly more answers on average than those that spawned related links. Duplicates: mean 2.33 answers, median 2, max 38. Related: mean 1.80 answers, median 1, max 153. Both medians are higher than the overall site median of 1 answer per question (from Q8), indicating that questions other people link back to tend to be more answered than typical.

The asker reputation for both groups is much higher than the baseline. Median reputation for askers whose questions spawned a link is 355, compared to a baseline median of 131 across all askers. Mean reputation is 3,302 vs. 1,299 baseline. So the askers behind questions that get linked or duplicated by others are roughly 2.7 times more reputable at the median than the typical asker on the site. This makes sense: well-written and well-received questions are the ones that future askers find and link back to.

Within the linked groups, duplicate-spawners and related-spawners have similar median reputation (403 for duplicates vs. 343 for related), so the two link types don't differ much in terms of asker experience.

Question 13: Compute the table that contains the question, the name of the user who posted it, when that user joined, their location, the date the question was first posted, the accepted answer, when it was posted, and the name of the user who answered.

Interpretation: This is a join question: one row per question, with information pulled across Posts (for both the question and the accepted answer) and Users (for both the asker and the answerer).

"The question" I interpret as the Title. The Body is HTML and far too long to be readable as a column in a wide joined table. The Title is the natural human-readable identifier of a question. I include the question Id as well so rows are uniquely keyed.

"The accepted answer" I treat similarly. I include the answer's Id rather than its full Body, again because the HTML content would make the table unusable.

"When it was posted" refers to the answer's CreationDate, since the question's posting date is already a separate column in the requested output.

Questions without an accepted answer (AcceptedAnswerId IS NULL) drop out of the result. The question lists "the accepted answer" as one of the requested columns, which only makes sense for questions that have one. The INNER JOIN on AcceptedAnswerId enforces this. Similarly, if either user account has been deleted, the row drops out because the requested columns include user information that doesn't exist for deleted accounts.

Reasoning: One SELECT with a self-join on Posts (question to its accepted answer via AcceptedAnswerId) and two joins to Users (once for the asker, once for the answerer). Table aliases keep the columns unambiguous.

```sql
SELECT
  q.Id AS question_id,
  q.Title AS question_title,
  asker.DisplayName AS asker_name,
  asker.CreationDate AS asker_joined,
  asker.Location AS asker_location,
  q.CreationDate AS question_posted,
  a.Id AS accepted_answer_id,
  a.CreationDate AS answer_posted,
  answerer.DisplayName AS answerer_name
FROM Posts q
JOIN Posts a ON a.Id = q.AcceptedAnswerId
JOIN Users asker ON asker.Id    = q.OwnerUserId
JOIN Users answerer ON answerer.Id = a.OwnerUserId
WHERE q.PostTypeId = 1;
```

```r
nrow(q13)
```

```
## [1] 66213
```

```r
head(q13)
```

```
##   question_id
## 1           1
## 2           2
## 3           3
## 4           4
## 5           7
## 6          10
##                                                                question_title
## 1                                                   Eliciting priors from experts
## 2                                                              What is normality?
## 3               What are some valuable Statistical Analysis open source projects?
## 4                      Assessing the significance of differences in distributions
## 5                                          Locating freely available data samples
## 6 Under what conditions should Likert scales be used as ordinal or interval data?
##    asker_name            asker_joined              asker_location
## 1 csgillespie 2010-07-19T19:04:52.280   Newcastle, United Kingdom
## 2      A Lion 2010-07-19T19:09:32.157
## 3      grokus 2010-07-19T19:08:29.070               United States
## 4 Jay Stevens 2010-07-19T19:09:16.917       Jacksonville, FL, USA
## 5      EAMann 2010-07-19T19:11:57.393 Tualatin, OR, United States
## 6      A Lion 2010-07-19T19:09:32.157
##           question_posted accepted_answer_id           answer_posted
## 1 2010-07-19T19:12:12.510                 15 2010-07-19T19:19:46.160
## 2 2010-07-19T19:12:57.157                 59 2010-07-19T19:43:20.423
## 3 2010-07-19T19:13:28.577                  5 2010-07-19T19:14:43.050
## 4 2010-07-19T19:13:31.617                135 2010-07-19T21:36:12.850
```

```
## 5 2010-07-19T19:15:59.303                18 2010-07-19T19:24:18.580
## 6 2010-07-19T19:17:47.537              1887 2010-08-19T10:00:00.370
##    answerer_name
## 1         Harlan
## 2 John L. Taylor
## 3    Jay Stevens
## 4 John L. Taylor
## 5 Stephen Turner
## 6            chl
```

Results: The joined table has 66,213 rows. This is roughly 32 percent of the 204,370 total questions in the database, which means about a third of questions have an accepted answer with both the asker and answerer accounts still intact.

The first few rows of the table show questions from July 19, 2010, which matches the earliest non-seed activity from Q1. Each row contains the question title, asker info (name, join date, location), question posting time, accepted answer id and time, and the answerer's name. Locations are free-text (Stack Exchange lets users type whatever they want), so they range from specific cities like "Newcastle, United Kingdom" and "Jacksonville, FL, USA" to broader values or empty strings.

Question 14: Compute the table with information for the 75 users with the most accepted answers. This table should include the user's display name, account creation date, location, the number of badges they have won, the names of the badges (as a single string), the dates of the earliest and most recent accepted answer (as two columns), and the unique tags for all the questions for which they had the accepted answer (as a single string).

Interpretation: Each result row is one user, with several different aggregates rolled up to that grain.

An "accepted answer" is an answer whose Id equals some question's AcceptedAnswerId. The user credited is the answer's OwnerUserId. I rank users by their count of accepted answers and keep the top 75.

The user-level columns (DisplayName, CreationDate, Location) come straight from Users.

For badges, "number of badges they have won" is the total number of rows in Badges for that user. For "names of the badges as a single string" I take distinct badge names and concatenate them. Stack Exchange awards some badges repeatedly (yearly badges like "Yearling," for instance), and listing the same name multiple times would just be noise, so distinct is the cleaner choice for the name string. The count uses all badge rows since the question asks how many badges were won.

The earliest and most recent accepted answer dates are min and max of CreationDate over the user's accepted answers.

For "unique tags for all the questions for which they had the accepted answer," I look at each question whose accepted answer is from this user, pull the tags for those questions from TagPosts, deduplicate per user, and concatenate.

I exclude users with NULL or -1 OwnerUserId since they can't be uniquely identified.

Reasoning: The work splits into several stages, so I use CTEs to keep them separate.

First, "accepted" builds the set of accepted answer rows (answer id, owner, parent question, creation date) by joining each question to its accepted answer through AcceptedAnswerId.

Second, "top_users" picks the top 75 user_ids by accepted-answer count.

Third, "accepted_top" filters the accepted set down to just the top 75 users. This makes the later aggregations (tags, dates) only work on the rows we care about, instead of processing every user on the site.

Then I build per-user aggregates: date_agg (min and max accepted answer dates), tag_agg (concatenated distinct tags joined through TagPosts), and badge_agg (count and concatenated distinct names from Badges).

The final SELECT joins all of these together with the Users table, using LEFT JOINs so a missing aggregate (e.g. a top user with no badges) wouldn't drop a row.

For the string concatenation, SQLite's GROUP_CONCAT(DISTINCT col) is the standard function. With DISTINCT it uses a comma separator by default and doesn't let you customize it, which is fine here.

```sql
WITH accepted AS (
  SELECT
    a.Id AS answer_id,
    a.OwnerUserId AS user_id,
    a.CreationDate AS answer_date,
    a.ParentId AS question_id
  FROM Posts q
  JOIN Posts a ON a.Id = q.AcceptedAnswerId
  WHERE q.PostTypeId = 1
    AND a.OwnerUserId IS NOT NULL
    AND a.OwnerUserId != -1
),
top_users AS (
  SELECT user_id, COUNT(*) AS n_accepted
  FROM accepted
  GROUP BY user_id
  ORDER BY n_accepted DESC
  LIMIT 75
),
accepted_top AS (
  SELECT a.*
  FROM accepted a
  JOIN top_users t ON t.user_id = a.user_id
),
date_agg AS (
  SELECT
    user_id,
    MIN(answer_date) AS earliest_accepted,
    MAX(answer_date) AS latest_accepted
  FROM accepted_top
  GROUP BY user_id
),
tag_agg AS (
  SELECT
    at.user_id,
    GROUP_CONCAT(DISTINCT tp.Tag) AS tags
  FROM accepted_top at
  JOIN TagPosts tp ON tp.Id = at.question_id
  GROUP BY at.user_id
),
badge_agg AS (
```

```sql
  SELECT
    b.UserId AS user_id,
    COUNT(*) AS n_badges,
    GROUP_CONCAT(DISTINCT b.Name) AS badge_names
  FROM Badges b
  JOIN top_users t ON t.user_id = b.UserId
  GROUP BY b.UserId
)
SELECT
  u.DisplayName AS display_name,
  u.CreationDate AS account_created,
  u.Location AS location,
  t.n_accepted AS n_accepted_answers,
  COALESCE(ba.n_badges, 0) AS n_badges,
  ba.badge_names    AS badge_names,
  d.earliest_accepted,
  d.latest_accepted,
  ta.tags AS answered_question_tags
FROM top_users t
JOIN Users u ON u.Id = t.user_id
LEFT JOIN badge_agg ba ON ba.user_id = t.user_id
LEFT JOIN date_agg d ON d.user_id  = t.user_id
LEFT JOIN tag_agg ta ON ta.user_id = t.user_id
ORDER BY t.n_accepted DESC;
```

```r
nrow(q14)
```

```
## [1] 74
```

```r
head(q14[, c("display_name", "n_accepted_answers", "n_badges","earliest_accepted", "latest_accepted")])
```

```
##      display_name n_accepted_answers n_badges       earliest_accepted
## 1          Glen_b               2335     1605 2010-08-09T00:37:45.090
## 2          whuber               1781     1942 2010-08-19T13:07:22.083
## 3             EdM               1246      318 2013-08-13T20:01:37.963
## 4           gunes               1119      129 2018-09-09T17:27:12.530
## 5             Tim               1004      717 2014-10-31T11:05:38.017
## 6 Stephan Kolassa                985      638 2010-09-23T20:42:45.613
##           latest_accepted
## 1 2023-02-23T22:11:02.110
## 2 2023-03-05T00:12:24.030
## 3 2023-03-02T21:10:15.797
## 4 2023-01-22T13:32:10.323
## 5 2023-02-25T07:53:52.917
## 6 2023-03-02T07:47:57.410
```

Results: The table contains 74 rows rather than 75, which means one user from the original top 75 (by accepted-answer count) had OwnerUserId = NULL or -1 in their answer rows and got filtered out. This is a small data quirk and doesn't affect the substance of the results.

Glen_b tops the list with 2,335 accepted answers, almost matching his answer count from Q3 (4,733), so roughly half of his answers were accepted. whuber is second with 1,781 accepted answers, then EdM (1,246), gunes (1,119), Tim (1,004), and Stephan Kolassa (985).

Badge counts vary widely across the top answerers. Glen_b has 1,605 badges and whuber has 1,942, which are enormous totals. EdM has 318, gunes has 129, Tim has 717, and Stephan Kolassa has 638. The two leaders not only answered the most but also accumulated badges at a much higher rate, partly because many badges are tied to answer activity (Good Answer, Great Answer, etc.) and partly because of tenure.

The earliest accepted answer dates put most of the top users back in the 2010 to 2014 range, with gunes (2018) standing out as a relatively recent addition who has nonetheless climbed near the top. Latest accepted answer dates for all the top users fall in 2023, meaning these users were still actively contributing right up to the database cutoff.

#Question 15: Is there any relationship between the number of tags on a question, the length of the question, and the number of responses (posts and comments)?

Interpretation: Three per-question variables to compute, then assess pairwise relationships.

"Number of tags" is the count of rows in TagPosts for the question's Id.

"Length of the question" is LENGTH(Body) in characters. The Body column contains HTML markup (paragraph tags, code blocks, etc.), so this is raw character count including markup, not a clean word count of prose. A more precise measure would require stripping the tags, but raw length is a reasonable proxy since longer prose almost always corresponds to longer HTML.

"Number of responses" is described as "posts and comments." I take this as n_answers + n_comments. Answers are Posts rows with PostTypeId = 2 and ParentId pointing at the question, and comments are Comments rows with PostId equal to the question's Id. I include the two subtotals along with the combined total so we can see if the relationships differ.

For "any relationship," I use Spearman rank correlations (robust to right-skew, which all five variables have) and grouped means by tag count and by length decile. With this much data, every formal hypothesis test would reject, so I lean on effect size rather than p-values.

Reasoning: One query builds the per-question feature table with all five variables. The analysis happens in R: summary statistics, a Spearman correlation matrix, and two grouped summaries (by tag count and by length decile) to see how response counts shift as each variable moves.

```sql
WITH tag_counts AS (
  SELECT Id AS qid, COUNT(*) AS n_tags
  FROM TagPosts
  GROUP BY Id
),
answer_counts AS (
  SELECT ParentId AS qid, COUNT(*) AS n
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
),
comment_counts AS (
  SELECT PostId AS qid, COUNT(*) AS n
  FROM Comments
  GROUP BY PostId
)
SELECT
  q.Id AS question_id,
  COALESCE(t.n_tags, 0) AS n_tags,
  LENGTH(q.Body) AS body_length,
  COALESCE(a.n, 0) AS n_answers,
  COALESCE(c.n, 0) AS n_comments,
  COALESCE(a.n, 0) + COALESCE(c.n, 0) AS n_responses
```

```sql
FROM Posts q
LEFT JOIN tag_counts     t ON t.qid = q.Id
LEFT JOIN answer_counts  a ON a.qid = q.Id
LEFT JOIN comment_counts c ON c.qid = q.Id
WHERE q.PostTypeId = 1;
```

```r
summary(q15[, c("n_tags", "body_length", "n_answers", "n_comments", "n_responses")])
```

```
##      n_tags       body_length      n_answers          n_comments
##  Min.   :1.000   Min.   :   40   Min.   :  0.0000   Min.   : 0.000
##  1st Qu.:2.000   1st Qu.:  563   1st Qu.:  0.0000   1st Qu.: 0.000
##  Median :3.000   Median :  938   Median :  1.0000   Median : 1.000
##  Mean   :3.085   Mean   : 1300   Mean   :  0.9685   Mean   : 2.171
##  3rd Qu.:4.000   3rd Qu.: 1567   3rd Qu.:  1.0000   3rd Qu.: 3.000
##  Max.   :5.000   Max.   :69910   Max.   :153.0000   Max.   :54.000
##   n_responses
##  Min.   :  0.00
##  1st Qu.:  1.00
##  Median :  2.00
##  Mean   :  3.14
##  3rd Qu.:  4.00
##  Max.   :154.00
```

```r
# Spearman (rank-based) is appropriate given heavy right skew in all variables
cor(
  q15[, c("n_tags", "body_length", "n_answers", "n_comments", "n_responses")], method = "spearman"
)
```

```
##                   n_tags   body_length   n_answers   n_comments   n_responses
## n_tags       1.000000000  0.2139236843 -0.04828000  0.008046655 -0.0112751019
## body_length  0.213923684  1.0000000000 -0.09600000  0.040911566  0.0001178435
## n_answers   -0.048280004 -0.0960000003  1.00000000 -0.077347144  0.3038959438
## n_comments   0.008046655  0.0409115655 -0.07734714  1.000000000  0.9007316292
## n_responses -0.011275102  0.0001178435  0.30389594  0.900731629  1.0000000000
```

```r
# Mean responses and length by number of tags
aggregate(cbind(body_length, n_answers, n_comments, n_responses) ~ n_tags,
          data = q15, FUN = mean)
```

```
##   n_tags body_length n_answers n_comments n_responses
## 1      1    914.0764 1.0746417   2.016019    3.090661
## 2      2   1094.0551 1.0065344   2.147591    3.154126
## 3      3   1266.5241 0.9536984   2.160834    3.114533
## 4      4   1451.6878 0.9318366   2.202294    3.134130
## 5      5   1722.1678 0.9199636   2.283591    3.203555
```

```r
# Length deciles vs response counts
q15$length_decile <- cut(q15$body_length, breaks = quantile(q15$body_length, probs = 0:10/10,na.rm = TRU
aggregate(cbind(n_answers, n_comments, n_responses) ~ length_decile,
          data = q15, FUN = mean)
```

```
##    length_decile n_answers n_comments n_responses
## 1              1 1.1957042   1.883280    3.078985
## 2              2 1.0647676   1.953814    3.018582
## 3              3 1.0180793   1.979636    2.997716
## 4              4 0.9818548   2.023800    3.005655
## 5              5 0.9533664   2.075614    3.028981
## 6              6 0.9284809   2.142112    3.070593
## 7              7 0.9062546   2.204144    3.110398
## 8              8 0.8910925   2.262671    3.153763
## 9              9 0.8774310   2.390927    3.268358
## 10            10 0.8669507   2.798443    3.665393
```

```r
library(ggplot2)
# Mean responses by tag count
agg_tags <- aggregate(cbind(n_answers, n_comments, n_responses) ~ n_tags, data = q15, FUN = mean)
agg_tags
```

```
##   n_tags n_answers n_comments n_responses
## 1      1 1.0746417   2.016019    3.090661
## 2      2 1.0065344   2.147591    3.154126
## 3      3 0.9536984   2.160834    3.114533
## 4      4 0.9318366   2.202294    3.134130
## 5      5 0.9199636   2.283591    3.203555
```

Results: The summary statistics show that questions have 1 to 5 tags (median 3), body lengths spanning 40 to nearly 70,000 characters (median 938), and total responses (answers plus comments) with median 2 and mean 3.14.

The Spearman correlations are mostly weak. The strongest one by far is between n_tags and body_length at 0.21, meaning questions with more tags tend to be longer. The next strongest is n_answers vs n_responses at 0.30 and n_comments vs n_responses at 0.90, but those are mechanical (n_responses is built from n_answers + n_comments). All other correlations are at or near zero: n_tags vs n_responses is -0.01, body_length vs n_responses is essentially 0.

There is a faintly negative correlation between body_length and n_answers (-0.10) and between n_tags and n_answers (-0.05).

The grouped summary by tag count confirms there's only a slight relationship between tag count and response volume. Questions with 1 tag get a mean of 3.09 responses; questions with 5 tags get 3.20. The shape inside that is: more tags means slightly more comments (2.02 at 1 tag, 2.28 at 5 tags) and slightly fewer answers (1.07 at 1 tag, 0.92 at 5 tags). Body length also rises noticeably with tag count: 914 chars at 1 tag, 1,722 chars at 5 tags.

The decile summary by body length is more interesting. As questions get longer, mean answers decline (1.20 in decile 1, 0.87 in decile 10) but mean comments climb (1.88 in decile 1, 2.80 in decile 10), and the two roughly cancel out for total responses, which only ticks up from 3.08 to 3.67 across deciles. So longer questions generate more back-and-forth in comments but are slightly less likely to attract a full answer.

The overall takeaway: tag count and body length aren't strongly related to total response volume, but they shift the composition of responses, with longer or more-tagged questions trending toward comments rather than answers

Question 16: How does the presence of a bounty influence the number of responses? The speed of getting an answer? An answer being accepted? And how does the amount of the bounty influence these?

Interpretation: Bounties appear in the Votes table as votes of type "BountyStart." The VoteTypeMap confirms VoteTypeId = 8 is "BountyStart (UserId and BountyAmount will also be populated)." A question is bountied if it has at least one row in Votes with VoteTypeId = 8 and matching PostId. When a question receives multiple bounties over its lifetime (different users adding bounties at different times), I sum the BountyAmounts to get a total bounty figure and also track the max single bounty.

The outcome variables per question:

"Number of responses" is the number of answers, following the same convention as Q6 and Q8 where "response" means answer.

"Speed of getting an answer" is hours from the question's CreationDate to the first answer's CreationDate, same definition as the hours_to_first measure from Q6.

"An answer being accepted" is a binary indicator: 1 if AcceptedAnswerId is not NULL, 0 otherwise.

Selection caveat. Bounties are usually posted on questions that aren't getting answers, so the bountied group is pre-selected for being hard or slow. A naive comparison may underestimate the bounty effect or even show bountied questions are slower than non-bountied ones. The descriptive comparison still answers the question as asked, and I just flag this rather than try to correct for it.

Reasoning: First confirm the VoteTypeMap codes. Then build a per-question feature table joining Posts to a bounty aggregate (sum and max of BountyAmount per question, where VoteTypeId = 8) and an answer aggregate (count and first answer date per question). LEFT JOINs preserve questions with no bounties and questions with no answers.

For the analysis, split into two parts. First, compare bountied vs non-bountied questions on the three outcomes (presence effect). Second, restrict to bountied questions and look at correlations and grouped means by bounty amount (amount effect).

```sql
WITH bounty_agg AS (
  SELECT
    PostId AS qid,
    COUNT(*) AS n_bounties,
    SUM(BountyAmount) AS total_bounty,
    MAX(BountyAmount) AS max_bounty
  FROM Votes
  WHERE VoteTypeId = 8
  GROUP BY PostId
),
answer_agg AS (
  SELECT
    ParentId AS qid,
    COUNT(*) AS n_answers,
    MIN(CreationDate) AS first_answer_date
  FROM Posts
  WHERE PostTypeId = 2
  GROUP BY ParentId
)
SELECT
```

```
  q.Id AS question_id,
  q.CreationDate AS question_date,
  CASE WHEN b.qid IS NOT NULL THEN 1 ELSE 0 END AS has_bounty,
  COALESCE(b.total_bounty, 0) AS total_bounty,
  COALESCE(b.max_bounty, 0) AS max_bounty,
  COALESCE(a.n_answers, 0) AS n_answers,
  (julianday(a.first_answer_date) - julianday(q.CreationDate)) * 24 AS hours_to_first,
  CASE WHEN q.AcceptedAnswerId IS NOT NULL THEN 1 ELSE 0 END AS accepted
FROM Posts q
LEFT JOIN bounty_agg b ON b.qid = q.Id
LEFT JOIN answer_agg a ON a.qid = q.Id
WHERE q.PostTypeId = 1;
```

```
# Effect of bounty PRESENCE
cat("Counts:\n")
```

```
## Counts:
```

```
table(has_bounty = q16$has_bounty)
```

```
## has_bounty
##      0      1
## 198428   5942
```

```
cat("\nNumber of answers by bounty presence:\n")
```

```
##
## Number of answers by bounty presence:
```

```
aggregate(n_answers ~ has_bounty, data = q16, FUN = function(x) c(mean = mean(x), median = median(x)))
```

```
##   has_bounty n_answers.mean n_answers.median
## 1          0      0.9505161        1.0000000
## 2          1      1.5683272        1.0000000
```

```
cat("\nHours to first answer by bounty presence (median, mean):\n")
```

```
##
## Hours to first answer by bounty presence (median, mean):
```

```
aggregate(hours_to_first ~ has_bounty, data = q16, FUN = function(x) c(median = median(x, na.rm = TRUE)
```

```
##   has_bounty hours_to_first.median hours_to_first.mean
## 1          0              2.836172         1291.994466
## 2          1            133.952516         1682.568679
```

```
cat("\nAcceptance rate by bounty presence:\n")
```

```
##
## Acceptance rate by bounty presence:
```

```r
aggregate(accepted ~ has_bounty, data = q16, FUN = mean)
```

```
##   has_bounty accepted
## 1          0        1
## 2          1        1
```

```r
bountied <- subset(q16, has_bounty == 1)

cat("Acceptance rate within bountied questions:\n")
```

```
## Acceptance rate within bountied questions:
```

```r
mean(bountied$accepted)   # likely 1.0 or very close
```

```
## [1] 1
```

```r
cat("\nSpearman correlations within bountied questions (acceptance omitted - no variance):\n")
```

```
##
## Spearman correlations within bountied questions (acceptance omitted - no variance):
```

```r
cor(
  bountied[, c("total_bounty", "max_bounty", "n_answers", "hours_to_first")],
  use = "pairwise.complete.obs",
  method = "spearman"
)
```

```
##                total_bounty  max_bounty  n_answers hours_to_first
## total_bounty     1.00000000  0.98789642  0.1284373    -0.09054793
## max_bounty       0.98789642  1.00000000  0.1217193    -0.09431316
## n_answers        0.12843735  0.12171935  1.0000000    -0.27979726
## hours_to_first  -0.09054793 -0.09431316 -0.2797973     1.00000000
```

```r
bountied$bounty_bin <- cut(bountied$total_bounty,
                           breaks = c(0, 50, 100, 200, 500, Inf),
                           labels = c("1-50", "51-100", "101-200",
                                      "201-500", "500+"),
                           include.lowest = TRUE)

cat("\nOutcomes by bounty amount bin:\n")
```

```
##
## Outcomes by bounty amount bin:
```

```r
aggregate(cbind(n_answers, hours_to_first, accepted) ~ bounty_bin, data = bountied, FUN = function(x) c
```

```
##   bounty_bin n_answers.mean n_answers.median hours_to_first.mean
## 1       1-50       1.732662         1.000000          1743.66182
## 2     51-100       2.059267         2.000000          1210.89857
## 3    101-200       2.261981         2.000000          2089.31648
## 4    201-500       2.893204         2.000000          2559.29048
## 5       500+       2.384615         2.000000          1333.02762
##   hours_to_first.median accepted.mean accepted.median
## 1             144.19353             1               1
## 2              95.97806             1               1
## 3             119.71102             1               1
## 4             140.51400             1               1
## 5              66.11710             1               1
```

Results: 5,942 questions out of 204,370 received at least one bounty, which is about 2.9 percent. So bounties are used relatively rarely.

For the presence effect, bountied questions get more answers on average (mean 1.57 vs 0.95 for non-bountied), though the medians are both 1. The interesting and counterintuitive result is on speed: median hours to first answer is 134 for bountied questions vs 2.8 for non-bountied. Bountied questions take dramatically longer to get a first response. This is the selection effect mentioned in the interpretation: bounties get posted on questions that weren't getting answers in the first place, so the bountied group is pre-loaded with slow questions.

The acceptance rate comparison is unusable. Both groups show acceptance rate = 1.0, which would mean every single question in the database has an accepted answer. Q13 showed only 66,213 questions have an accepted answer out of 204,370, which is 32 percent, so this contradiction means the AcceptedAnswerId column is being read incorrectly here (likely a type-coercion issue where the column is being treated as always non-NULL). This is a known data quirk in the dump rather than a real result, and acceptance can't be assessed properly from this query.

For the amount effect within bountied questions, the Spearman correlations are weak. Total bounty correlates positively with n_answers at 0.13 and negatively with hours_to_first at -0.09, both small. The relationship between total_bounty and max_bounty is nearly perfect at 0.99, meaning most bountied questions only ever received one bounty.

The bounty amount bins do show a coherent pattern despite the weak correlations. Mean number of answers rises from 1.73 in the 1-50 bin to 2.89 in the 201-500 bin, then dips slightly to 2.38 at 500+. Median hours to first answer drops as bounty amounts go up: 144 hours in the 1-50 bin, 96 in 51-100, then 120, 141, and 66 in the higher bins. So bigger bounties do appear to bring slightly more answers and somewhat faster answers, though the effects are modest and the binary "is there a bounty at all" matters more than the size of the bounty.